

PROJECT TECHNICAL DOCUMENTATION

Proj_Doc • Neural Overlay • SyncCash

Comprehensive developer, setup, execution, architecture, code-walkthrough and troubleshooting guide

Author: Aniket Joshi

Repository owner: Aniket15112005

Prepared: 16 August 2026

This document is based on the public GitHub repositories and their current main-branch source/README/configuration files available on 16 August 2026. It explains what the code actually implements; it does not invent unverified functionality.

2. Common Development Environment

2.1 Recommended Windows setup

For the two Flutter projects, install Flutter SDK, Dart (bundled with Flutter), Android Studio/Android SDK if Android execution is required, Git, Node.js/npm for Firebase CLI, and VS Code or another Flutter-capable IDE. For Neural Overlay, install Python 3.9+ and FFmpeg.

Verify Flutter and Dart

```
flutter --version
dart --version
flutter doctor
flutter devices
```

Verify Git and Node/npm

```
git --version
node --version
npm --version
```

Verify Python and FFmpeg

```
python --version
ffmpeg -version
```

2.2 PowerShell vs CMD

Flutter and Git commands are essentially identical in PowerShell and CMD. Python virtual-environment activation differs slightly only in shell syntax.

Task	PowerShell	CMD
Create venv	python -m venv venv	python -m venv venv
Activate	.\venv\Scripts\Activate.ps1	venv\Scripts\activate.bat
Install Python deps	pip install -r requirements.txt	pip install -r requirements.txt
Run Python	python script.py	python script.py

2.3 General Git workflow

Clone and enter a repository

```
git clone <repository-url>
cd <repository-folder>
git status
git pull
```

Do not commit generated build folders, local environment files, private credentials, IDE state, or machine-specific artifacts. Both Flutter repositories contain generated/platform-specific material, so the tracked configuration should be treated as the source of truth.

5. SyncCash — Technical Documentation

SyncCash is a real-time collaborative cashbook and micro-ledger platform built with Flutter and Firebase. Its README describes shared business cashbooks, invite-based participation, live synchronization, income/expense transactions, financial dashboards, audit/history and Firebase-backed infrastructure. ■cite■turn5view2■

5.1 Functional scope

- Firebase Authentication with email/password and Google Sign-In support in the dependency set.
- Create and join business cashbooks using invite codes.
- Real-time shared ledger synchronization.
- Income and expense transactions with categories and descriptions.
- Running balance and dashboard totals.
- Transaction history and user attribution.
- Bills, purchases, daily entries, reports, analytics, chatbot, notifications and supporting infrastructure are represented in the current repository structure.

The current repository is substantially larger than the concise README structure: lib contains app, core, features, shared and voice layers, while features include analytics, auth, bills, cashbook, chatbot, daily, dashboard, purchases, reports, sales, settings and transactions. ■cite■turn3view1■turn7view2■

5.2 Technology stack and dependencies

The pubspec declares Flutter/Dart SDK >=3.3.0 <4.0.0, Riverpod, GoRouter, Firebase Core/Auth/Firestore/Storage/Messaging, Google Sign-In, local notifications, speech-to-text, audio recording, HTTP, dotenv, charts, PDF/Excel export and file utilities. ■cite■turn3view2■

Layer	Key technologies
UI	Flutter Material, flutter_animate, fl_chart
State	flutter_riverpod / Riverpod annotations
Navigation	GoRouter
Identity	Firebase Auth, Google Sign-In
Data	Cloud Firestore
Files	Firebase Storage, file_picker, path_provider
Notifications	Firebase Messaging, flutter_local_notifications
Voice	record, speech_to_text
Export	pdf, excel, share_plus, pdfx, flutter_pdfview
Config	flutter_dotenv

5.3 Repository architecture

High-level structure

```
lib/  
  app/  
    router/  
    theme/  
    app.dart  
  core/  
    errors/  
    navigation/  
    services/  
    utils/
```

```

features/
  analytics/
  auth/
  bills/
  cashbook/
  chatbot/
  daily/
  dashboard/
  purchases/
  reports/
  sales/
  settings/
  transactions/
shared/
voice/
firebase_options.dart
main.dart

functions/
  index.js
  package.json
  package-lock.json

```

The repository also contains Android/iOS/Web/Windows platform projects, Firebase hosting/functions configuration, deployment scripts and COMMANDS.md. [■cite■turn1view0■turn13view0■](#)

5.4 Startup sequence — main.dart

SyncCash performs a more extensive startup sequence than Proj_Doc: it loads app.env, initializes Firebase, configures Firestore persistence differently for mobile and web, starts a Firestore reconnection service, locks mobile orientation to portrait, applies system UI styling, runs the Riverpod-wrapped application, and initializes notifications after runApp so UI startup is not blocked. [■cite■turn11view0■](#)

Core startup sequence

```

await dotenv.load(fileName: 'app.env');

await Firebase.initializeApp(
  options: DefaultFirebaseOptions.currentPlatform,
);

if (!kIsWeb) {
  FirebaseFirestore.instance.settings = const Settings(
    persistenceEnabled: true,
    cacheSizeBytes: 100 * 1024 * 1024,
  );
} else {
  FirebaseFirestore.instance.settings = const Settings(
    persistenceEnabled: true,
    webExperimentalAutoDetectLongPolling: true,
  );
}

FirestoreReconnectService.instance.start();

runApp(const ProviderScope(child: SyncCashApp()));

NotificationService.initialize();

```

5.5 App shell

SyncCashApp is a ConsumerWidget. It watches FCM initialization and the app router, then creates MaterialApp.router with light/dark themes and the GoRouter configuration. The current app explicitly forces ThemeMode.dark.

[■cite■turn11view1■](#)

5.6 Firebase platform configuration

firebase_options.dart is generated by FlutterFire CLI. It selects the correct FirebaseOptions based on kIsWeb or TargetPlatform. The current configuration includes Android, iOS, Web and Windows; macOS and Linux currently throw UnsupportedError and instruct the developer to re-run FlutterFire. [■cite■turn11view2■](#)

5.7 Firestore data model

The README documents users, cashbooks and transactions as the core logical model. A user contains UID, display name, email and currentCashbookId. A cashbook contains invite code, owner/participant IDs and totals. Transactions contain IDs, cashbook/user attribution, amount, type, category, description and timestamp. ■cite■turn5view2■

Collection / path	Important fields / purpose
users/{userId}	Identity, display name/email, current cashbook, platform metadata
cashbooks/{cashbookId}	Owner, participant, invite code, aggregate totals
cashbooks/{id}/transactions/{txId}	Income/expense ledger entries
cashbooks/{id}/deleted_transactions/{id}	Recycle-bin / soft-delete records
cashbooks/{id}/sale_bills/{id}	Sale bill records
cashbooks/{id}/purchase_bills/{id}	Purchase records, restricted to iOS/PWA
cashbooks/{id}/daily_entries/{id}	Daily feature records, restricted to iOS/PWA
cashbooks/{id}/parties/{id}	Opening-balance / party records
cashbooks/{id}/custom_bills/{id}	Custom bill maker records

5.8 Real-time synchronization

The documented transaction path is: write transaction to Firestore → stream the update to connected users → apply it to running balances → reflect the result immediately in active sessions. ■cite■turn1view2■

Conceptual synchronization



5.9 Firestore security model

The current Firestore rules use isCashbookMember() to verify that the authenticated user is the cashbook owner or participant. They also define platform-aware restrictions through isLosUser(). Bank and CB transaction categories are restricted, Android users cannot delete active transactions, and iOS/PWA users have broader transaction and recycle-bin privileges. Purchase and Daily collections are also restricted to iOS/PWA/web. ■cite■turn12view3■

This is one of the most important parts of SyncCash to understand for a security review: authorization is enforced server-side in Firestore rules, not only by hiding UI controls.

5.10 Firebase infrastructure

firebase.json configures Firestore in asia-south2, points to firestore.rules, configures Flutter platform application IDs, defines Firebase Hosting for build/web with SPA rewrites and cache headers, and declares a Cloud Functions codebase under functions. ■cite■turn12view2■

5.11 Installation and basic execution

Fresh clone

```

git clone https://github.com/Aniket15112005/Synccash.git
cd Synccash

flutter clean
flutter pub get
flutter devices
flutter run
  
```

The README's installation section also uses flutter pub get and flutter run. ■cite■turn1view2■

5.12 Environment configuration

The application loads a file named app.env before Firebase initialization. pubspec.yaml declares app.env as an asset. Therefore, a fresh environment must provide the expected variables used by the repository's feature/service code. Do not copy private keys into the repository; keep app.env local and ignored unless a particular value is intentionally public and the architecture expects it.

PowerShell example

```
# From the project root
Copy-Item app.env.example app.env # if the repository provides an example
# Edit app.env and fill required values
flutter pub get
flutter run
```

If an app.env.example file is not present in the clone, inspect the source for dotenv.env['...'] lookups and create app.env with exactly those variable names. Do not guess API-key names.

5.13 Commands documented by the project

COMMANDS.md defines a clean dependency/preflight sequence, Android release App Bundle generation, iOS release compilation without code signing, Firebase CLI installation/login and Firestore deployment. ■cite■turn8view0■

Official project command sequence

```
flutter clean
flutter pub get

flutter build appbundle --release
flutter build ios --release --no-codesign
```

```
npm install -g firebase-tools
firebase login
firebase deploy --only firestore
```

Useful development commands

```
flutter analyze
flutter test
flutter run -d chrome
flutter build web --release
flutter build apk --release
```

5.14 Firebase deployment

CLI authentication and deployment

```
npm install -g firebase-tools
firebase login
firebase deploy --only firestore
```

For full hosting/functions deployment, inspect firebase.json and deploy the required targets deliberately rather than deploying every target blindly. The repository currently declares Hosting and Cloud Functions infrastructure.

■cite■turn12view2■

5.15 Notification architecture

main.dart starts NotificationService after runApp. The app shell watches fcmInitProvider, while core/services contains FCM service/provider files, notification implementations and a web/stub split. This indicates a platform-aware notification architecture intended to keep native notification logic separate from unsupported environments.

■cite■turn11view0■turn11view1■turn13view1■

5.16 Voice architecture

The dependency set includes record and speech_to_text, and the repository has a dedicated lib/voice directory. The intended architecture is therefore to isolate audio capture and speech recognition from the core financial domain, reducing coupling between voice input and ledger logic. ■cite■turn3view2■turn7view4■

5.17 Export/report capabilities

SyncCash includes PDF and Excel packages, file picking, PDF viewing and sharing dependencies. This supports a workflow where financial data can be transformed into reports, saved locally and shared. The exact screen/service responsible for each export should be followed from the feature module when modifying the implementation.

■cite■turn3view2■

5.18 End-to-end execution sequence

- Install Flutter/Dart, Android tooling if required, Node/npm and Firebase CLI.
- Clone SyncCash and run flutter clean followed by flutter pub get.
- Create/configure the local app.env required by the current source.
- Verify Firebase project configuration and platform application registrations.
- Authenticate the Firebase CLI.
- Deploy Firestore rules before testing privileged operations.
- Run the app on Android, Web or another configured target.
- Create/sign in to a user.
- Create a cashbook or join an existing cashbook with its invite mechanism.
- Create income and expense transactions and observe running totals.
- Open a second client/session to verify real-time synchronization.
- Test Firestore authorization by attempting operations as owner, participant and unsupported platform/category combinations.
- Test notifications, reports, exports and optional voice features separately.

7. Troubleshooting and Reproducibility

7.1 Flutter: dependency/build failures

Standard reset

```
flutter clean
flutter pub get
flutter doctor -v
flutter analyze
flutter run
```

- If an Android build fails, verify Android SDK/Gradle/JDK compatibility using `flutter doctor -v`.
- If a Firebase initialization error appears, regenerate `firebase_options.dart` with `FlutterFire`.
- If a web build behaves differently from mobile, check `kisWeb` branches and browser persistence restrictions.

7.2 Firebase: permission-denied

- Verify the user is authenticated.
- Verify document `userId/cashbookId` fields match the authenticated identity.
- Deploy the latest `firestore.rules`.
- Use Firebase Emulator Suite for safe rule testing.
- For `SyncCash`, verify the user is `ownerId` or `participantId` of the requested cashbook.

`SyncCash`'s rule model explicitly uses membership checks for cashbooks and transactions. [■cite■turn12view3■](#)

7.3 Firestore index errors

`Proj_Doc` uses ordered queries such as `userId + updatedAt` and `projectId + userId + createdAt`. The repository tracks composite indexes specifically to make these queries reproducible. Deploy indexes before assuming the application code is broken. [■cite■turn1view5■](#)

Deploy indexes

```
firebase deploy --only firestore:indexes
```

7.4 Neural Overlay: FFmpeg not found

Windows verification

```
where.exe ffmpeg
ffmpeg -version
```

- Install FFmpeg and add its bin directory to `PATH`.
- Restart the terminal after changing `PATH`.
- Run `ffmpeg -version` before running the Python script.

`hd_enhance.py` explicitly checks `shutil.which('ffmpeg')` and exits with an error when FFmpeg is missing. [■cite■turn5view1■](#)

7.5 Neural Overlay: poor tracking

- Use the recommended enhancement stage first for noisy/low-quality footage.
- Lower `CORNER_QUALITY` to detect more features.
- Lower `REFRESH EVERY` for faster-changing motion.
- Increase `MAX_POINTS` for a denser graph.
- Use footage with adequate contrast and motion.
- Remember that the tracker follows visual features, not semantic objects.

7.6 SyncCash: stuck loading / realtime connection

The current startup explicitly starts FirestoreReconnectService and configures persistence. The source comments explain that the web cache configuration was changed to avoid conflicting cache/persistence initialization and cold-launch behavior. ■cite■turn11view0■

Useful diagnostic sequence

```
flutter clean
flutter pub get
flutter run -d chrome
# or
flutter run
```

- Check browser console for Firestore failed-precondition errors.
- Check Firebase project/Firestore availability.
- Confirm the authenticated user's cashbook membership.
- Test with a second client to distinguish local UI state from backend synchronization.

7.7 Release validation checklist

- flutter doctor -v passes required platform checks.
- flutter analyze completes without blocking errors.
- flutter test passes.
- Firebase rules are deployed and tested.
- All required Firebase platform apps exist.
- Local environment files are present but not committed.
- A clean clone can reproduce dependency installation.
- Release APK/App Bundle/Web build is generated successfully.
- Authentication works on the intended release platform.
- Realtime synchronization is verified with two clients.

8. Security and Maintenance Recommendations

8.1 Proj_Doc

- Keep Firebase security rules deployed and version-controlled.
- Do not weaken rules to 'test mode' for production.
- Move third-party service configuration to controlled environment/configuration where appropriate.
- Validate Cloudinary upload restrictions and file types/size limits server-side where possible.
- Consider server-side search or indexed search if entry volume grows.
- Test deletion behavior to ensure cascading entry deletion is intentional.

8.2 Neural Overlay

- Treat input videos as potentially sensitive local data.
- Avoid uploading source videos unless explicitly required.
- Validate input/output paths and avoid overwriting originals by default.
- If processing untrusted media at scale, sandbox FFmpeg/OpenCV execution.
- Document CPU/RAM expectations for high-resolution videos.

8.3 SyncCash

- Treat Firestore rules as a core security control, not a configuration detail.
- Regression-test owner vs participant privileges and platform restrictions.
- Never put private API credentials into Dart source or public configuration.
- Protect app.env and ensure it is ignored by Git.
- Audit notification tokens, storage paths and callable/HTTP function authorization.
- Financial records should use server-authoritative timestamps and carefully controlled aggregate updates.
- Consider Cloud Functions or transactions for sensitive balance mutations where concurrent writes could otherwise create race conditions.

8.4 Credential/configuration note

Firebase web/mobile API keys appearing in generated `firebase_options.dart` are identifiers used by Firebase client SDKs and are not equivalent to server-side administrative credentials. Security still depends on Firebase Authentication, Firestore/Storage rules and backend authorization. Never commit Firebase Admin SDK service-account private keys, OAuth client secrets, database passwords or third-party private API tokens.

8.5 Documentation maintenance

Regenerate this documentation whenever a major feature, dependency, Firebase schema, security rule or deployment procedure changes. The most important files to re-check are `pubspec.yaml`, `main.dart`, `app/router` files, Firebase rules/configuration, environment variable definitions, service classes and deployment scripts.

9. Quick Reference — Commands

Proj_Doc

Clone → install → Firebase → run

```
git clone https://github.com/Aniket15112005/Proj_Doc.git
cd Proj_Doc
flutter pub get
```

```
dart pub global activate flutterfire_cli
npm install -g firebase-tools
firebase login
flutterfire configure
```

```
firebase deploy --only firestore:indexes,firestore:rules
```

```
flutter run
flutter run -d chrome
flutter build apk --release
```

Neural Overlay

Clone → venv → dependencies → execute

```
git clone https://github.com/Aniket15112005/Numpy_Overlay.git
cd Numpy_Overlay
```

```
python -m venv venv
.\venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

```
python hd_enhance.py input.mp4 clean.mp4
python node_graph_overlay_v2.py clean.mp4 output.mp4
```

SyncCash

Clone → dependencies → Firebase → run/build

```
git clone https://github.com/Aniket15112005/Synccash.git
cd Synccash
```

```
flutter clean
flutter pub get
```

```
npm install -g firebase-tools
firebase login
firebase deploy --only firestore
```

```
flutter run
flutter run -d chrome
flutter build apk --release
flutter build appbundle --release
flutter build web --release
```

Final execution philosophy

For all three projects, the safest way to reproduce a working environment is: start from a clean clone → verify prerequisites → install exact project dependencies → configure external services → deploy security/configuration → run a development build → exercise the main workflow → run static analysis/tests → build the release artifact. This sequence separates environment failures from application-code failures and makes debugging significantly faster.

Source basis

This document was prepared from the public repositories supplied by the developer, including README files, setup/command documentation, dependency manifests, Firebase configuration/rules and the principal application/source files. The repository descriptions and current structures were cross-checked against the GitHub sources available at preparation time.